

This homework has 8 questions, for a total of 100 points and 3 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up or any AI tools*.

(0 pts) 0. **Before you start; before you submit.**

- (a) Before starting this assignment, carefully read [Handout 3](#) about “giving an algorithm,” and follow it in your solutions.
- (b) Correctness arguments and runtime analysis for divide-and-conquer/recursive algorithms should follow the recommended structure described in this week’s discussion worksheet.
- (c) If applicable, state the name(s) and `username(s)` of your collaborator(s).

- (10 pts) 1. **Self assessment.** Carefully read and understand the posted solutions to the previous homework. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn’t show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn’t do last week’s homework, choose a problem from it that looks challenging to you, and in a few sentences, explain the key ideas behind its solution in your own words.

2. **Recurrences.**

Define $T(n) = 9T(n/3) + O(g(n))$. With brief justification, apply the Master Theorem to derive a closed-form asymptotic (big-O) solution for $T(n)$ when $g(n)$ is each of the following:

- (5 pts) (a) $g(n) = n^{3.5}$.
- (5 pts) (b) $g(n) = n \log n$.
- (5 pts) (c) $g(n) = 5n^2$.

(15 pts) 3. **This is the pits.**

A *pit* of an integer array $M[1, \dots, n]$ is an element $M[i]$ that is no larger than its adjacent elements. More precisely, $M[i]$ is a pit if $M[i] \leq M[i - 1]$ and $M[i] \leq M[i + 1]$, where for convenience we say that $M[0] = M[n + 1] = \infty$ to deal with the endpoints of the array. Note that by this definition, if M has just one element, then that element is a pit. Every array has at least one pit—namely, any smallest element in the array—though there may be others.

A trivial algorithm can find the index of a pit of $M[1, \dots, n]$ in $O(n)$ time. **Give an algorithm** to find the index of a pit in $O(\log n)$ time.

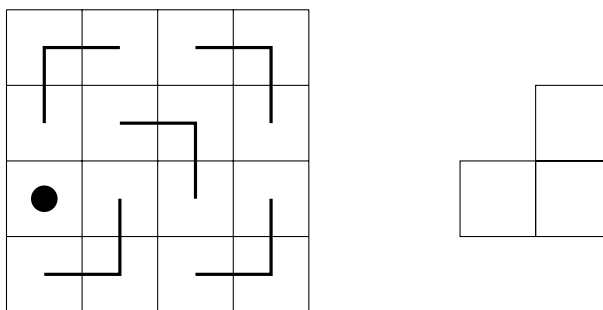
Hint: Think about another $O(\log n)$ -time algorithm you know that operates on one-dimensional arrays.

Note: Make sure you have read [Handout 3](#) before attempting this question, and that you include all of the required parts.

4. A hole different kind of tiling.

Consider the following tiling problem: for some $n \geq 2$ that is a power of two, we want to tile an n -by- n grid of unit squares using “L-shaped” tiles that are made up of three unit squares each. However, there is *one designated* unit square in the grid that should remain *uncovered*. The rest should be covered by tiles, without any overlap and with every tile contained entirely within the grid. The tiles may be oriented arbitrarily.

See the diagram below for a small example of a four-by-four grid and the shape of the tile. The uncovered square is marked by a circle, and the thick right angles show the orientations of the tiles in a valid tiling.



It is not immediately obvious that there is a valid tiling for any power-of-two n and desired uncovered square; indeed, certain approaches for placing tiles may “get stuck” in unresolvable configurations. In this problem you will develop and analyze an algorithm that works in all cases.

- (10 pts) (a) Below is an *incomplete* algorithm for this problem, whose missing parts you will provide. The input is a square n -by- n grid where $n \geq 2$ is a power of two, and (the location of) a designated unit square to leave uncovered. The algorithm places tiles accordingly on the grid; this tiling is its “output.”

Provide the missing parts to make the algorithm correct. (You will justify correctness and analyze running time in the subsequent parts.)

Algorithm outline (please fill in the blanks):

1. *Base case* (if $n =$ [MISSING PART #1]):
[MISSING PART #2]
2. *Recursive case* (if $n >$ [MISSING PART #3]):
 - Designate three additional unit squares, different from the given one, to leave uncovered for now, so that:
 - i. they can be covered by a single tile, and
 - ii. each of the four $n/2$ -by- $n/2$ quadrants (upper left, lower right, etc.) of the grid has exactly one square to leave uncovered.

Specifically, choose the three squares as follows: [MISSING PART #4]

- [MISSING PART #5]

- Place a tile to cover the three additional unit squares that were designated above.

- (10 pts) (b) Briefly justify the correctness of the algorithm, as you completed it in the previous part.
- (5 pts) (c) Let $T(n)$ denote the running time of the algorithm as a function of the side length n . The cost to place a tile at a specified location, with a specified orientation, is $O(1)$.
Derive, with brief justification, a recurrence and asymptotic closed-form solution for $T(n)$; both should be the tightest possible.

(15 pts) 5. **Hadamard Matrix multiplication.**

The matrix H_t is a $n \times n$ matrix where $n = 2^t$. It is defined recursively, where $H_0 = (1)$, and in general,

$$H_{t+1} = \begin{pmatrix} H_t & H_t \\ H_t & -H_t \end{pmatrix}$$

For example,

$$H_2 = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}$$

It turns out that this matrix has many [applications](#) including signal processing and quantum computation.

Given a vector $x \in \mathbb{Z}^n$, there is a trivial algorithm that uses $O(n^2)$ operations to compute the matrix-vector product $H_t x$, by first computing the matrix H_t and then computing its product with x . Give an algorithm that computes $H_t x$ in $O(n \cdot t) = O(n \log n)$ operations.

Note: Recall how matrix vector multiplication works. If we have a $n \times n$ matrix A and a vector $x \in \mathbb{Z}^n$, their product is calculated by taking the dot product of x with each row in A . (That

is, if the row is $[a_1, a_2, \dots, a_n]$ and $x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$, then the dot product is $a_1 x_1 + a_2 x_2 + \dots + a_n x_n$).

For example, here we have a 2×2 matrix A and a vector $x \in \mathbb{Z}^2$.

$$Ax = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} ax_1 + bx_2 \\ cx_1 + dx_2 \end{bmatrix}$$

Moreover, instead of multiplying each individual row and column, we can also multiply matrices in blocks. For example, if we have a 4×4 matrix A and a vector $x \in \mathbb{Z}^4$, we can break A into 4

smaller matrices, say 2×2 matrices H, I, J, K . We can also break x into 2 vectors $\vec{x}_1$ and $\vec{x}_2$ each of size 2. This would give the following formulation of the product Ax .

$$Ax = \begin{bmatrix} H & I \\ J & K \end{bmatrix} \begin{bmatrix} \vec{x}_1 \\ \vec{x}_2 \end{bmatrix} = \begin{bmatrix} H\vec{x}_1 + I\vec{x}_2 \\ J\vec{x}_1 + K\vec{x}_2 \end{bmatrix}.$$

- (20 pts) 6. **Maize-Blue-Coloring.** Let $G = (V, E)$ be an undirected graph. A *maize-blue* coloring on G is an assignment (denoted by color) from each vertex $v \in V$ to one of the colors maize or blue. Determine if the following algorithm terminates on all inputs.

```
1: function MAIZE-BLUE-COLORING( $G = (V, E)$ )  
   // Initialization  
2:   for  $v \in V$  do  
3:     Randomly assign either maize or blue to  $v$   
   // Main loop  
4:   while there exists a vertex  $v$  with more than half of  $v$ 's neighbors the same color as  $v$  do  
5:     Recolor  $v$  to the opposite color
```

If you think it terminates on every input, use the potential method to prove that the algorithm terminates, where your potential function may reference the vertices and edges of the graph, including information about how the vertices are colored.

If you think it does not terminate on every input, provide a graph and coloring on which the algorithm does not terminate and show the state of the graph at each step of the algorithm until the graph enters a state which it was in previously.

- (3 EC pts) 7. **Optional extra credit: Robot Testing!**

The Robotics Department has n supposedly identical robots that – in principle – are capable of testing each other. The Robotics Department's test jig accommodates two robots at a time. When the jig is loaded, each robot tests the other and reports whether it is good or bad. A good robot always reports accurately whether the other robot is good or bad, but the answer of a bad robot cannot be trusted - that is, *a bad robot may respond incorrectly or correctly*. For example, if two bad robots are placed in front of each other, it's possible (though not guaranteed) that the two robots will both falsely state that the other robot is good. Thus, the four possible outcomes of a test are as follows:

Robot A says -----	Robot B says -----	Conclusion -----
B is good	A is good	Both good, or both bad
B is good	A is bad	At least one is bad
B is bad	A is good	At least one is bad
B is bad	A is bad	At least one is bad

Assume that we are guaranteed that *more* than half of the n robots are good. Our ultimate goal is to identify all of the good robots. To do that, we'll use a divide-and-conquer process to find one good robot. Then, we'll use that one good robot to test each of the other $n - 1$ robots.

Here's how the divide-and-conquer step will work: We'll pair up the n robots arbitrarily (arbitrarily because we don't yet know anything about which robots are good and which are bad). If n is odd, one robot won't be paired. Then, for each pair of robots, we'll see what they say about one another. Based on what they say, we'll somehow decide which robots to advance to the next phase. Our goal is to advance $n/2$ robots or fewer to the second phase while maintaining the property that more than half of the robots in that second phase are good.¹ Then we'll do the same thing again. We'll pair up the robots that made it to the second phase (possibly leaving one unpaired if the number of robots is odd), see what they say about one another, and again somehow select at most half of those robots to go on to the third phase, maintaining the property that more than half of the robots in that phase are good. This is divide-and-conquer because we're dividing the population of robots by roughly a factor of two and then conquering the set of robots in a smaller set! At the end of this process, we'll get down to one robot that we know is good and we'll use that robot to test all of the original $n - 1$ other robots. Snazzy!

- (a) Given a set of n robots, more than half of which are good, explain how $n/2$ pairwise tests can be used to reduce that set to $n/2$ (or perhaps $\frac{n+1}{2}$) or fewer robots where, still, more than half of those robots are good. In your explanation, specify which robots are kept and which are discarded based on the outcomes of the pairwise tests. (Start off by assuming that n is even. Once you've worked out that case, come back and work on the case that n is odd.)
- (b) Using the result of part (a) above and still assuming that more than $\frac{n}{2}$ of the robots are good, give the tightest asymptotic bound for the worst case number of comparisons this algorithm must make to determine whether each robot is good or not.

¹When n is odd, it's fine if you let $\frac{n+1}{2}$ through to the next round. As long as $n > 1$, this will still reduce the number of robots. For simplicity, you may treat this as $n/2$ in your analysis